SNAPDRAGON AI CHALLENGE 2026 · ON-DEVICE GENERATIVE AI ACCELERATION & OPTIMIZATION

Ising-Edge

Budget-exact compression planning for on-device LLMs

Measure → fit → solve: measured perplexity, a quadratic surrogate, an exact solver



Example output: 18-MLP byte-budget plan, layers 0–27 (blue = keep 0.8, grey = dense, dark = not searched); full-split ΔPPL +62.99%

Jenish Patel

30 September 2026

THE PROBLEM

Every layer needs a compression decision

- Edge devices impose tight memory and bandwidth limits on small language models.
- Low-rank compression of MLP weights saves bytes, but its cost depends on *which* layers are compressed and *how strongly*.
- On Snapdragon, each weight tensor can also be placed on CPU, Adreno GPU or Hexagon NPU (llama.cpp tensor overrides).
- The decision is a **plan over all layers under a hard byte budget**: a combinatorial problem.

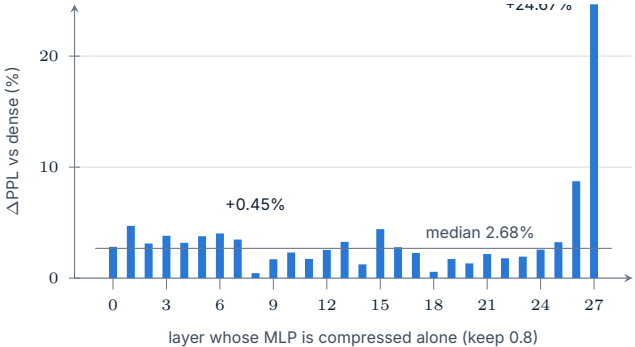
9.9×10^9

feasible plans for one budget: 24 MLP blocks (layers 2–25) × {dense, keep 0.9, keep 0.8}, bytes of 12 MLPs at keep 0.8

Plan count from isingedge/loop/space.py (Space.count); llama.cpp placement per docs/research/hetero-placement-2026.md.

EVIDENCE 1 · SINGLE-LAYER SENSITIVITY

Where you compress matters



+0.45% to +24.67%

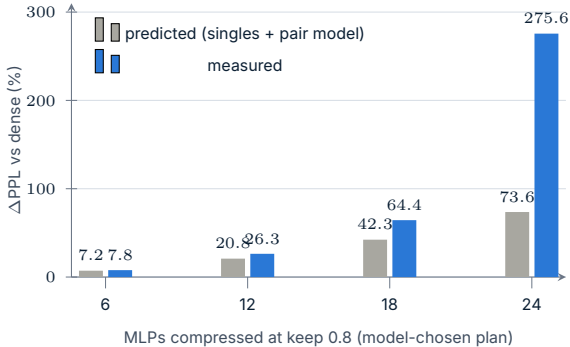
damage range across the 28 layers (layer 8 vs layer 27)

+2.68%

median layer at keep 0.8 (+1.68% at keep 0.9)

Activation-aware SVD of one MLP (rank 608); WikiText-2, 40 chunks × 512 tokens. results/sensitivity/qwen3-0.6b_layer_sensitivity.csv

Damage compounds: per-layer costs do not add up



+275.60% vs +57.75%

24 MLPs: measured vs summed single-layer costs (4.8×); log-additive +76.75%, pair model +73.62%

33 of 40 random layer pairs are super-additive (median +0.21 pp; largest +10.06 pp, layers 0+1).

Plans must be measured, not only predicted.

40 chunks. results/matched_bytes/summary.csv, results/sensitivity/qwen3-0.6b_pair_sensitivity.csv

OUR APPROACH

Ising-Edge: measure → fit → solve

1 Measure

Compress the plan's MLPs (activation-aware SVD) and score WikiText-2 perplexity on 40 fixed chunks; keep per-chunk NLL.

2 Fit

Quadratic surrogate of $\log(\mathrm{PPL}/\mathrm{PPL}_{\mathrm{dense}})$ over one-hot plan bits: pair model (one-shot) or rank-6 factorization machine (loop).

3 Solve

Minimise it exactly with CP-SAT: at most one option per layer, exact byte budget, no-good cuts on measured plans.

$$\min_{x\in\{0,1\}^{48}} \; w_0 + \sum_i w_i x_i + \sum_{i<j} \langle v_i, v_j \rangle x_i x_j \quad \text{s.t.} \sum_o x_{l,o} \le 1 \; \forall l, \;\; \sum_i u_i x_i = B$$

Output: per-layer keep fractions, re-measured on the full 584-chunk split with paired-bootstrap CIs. The same QUBO form admits latency (SnapTwin) and CPU/GPU/NPU placement terms: implemented, not yet validated on a device.

Implementation: what is built and tested

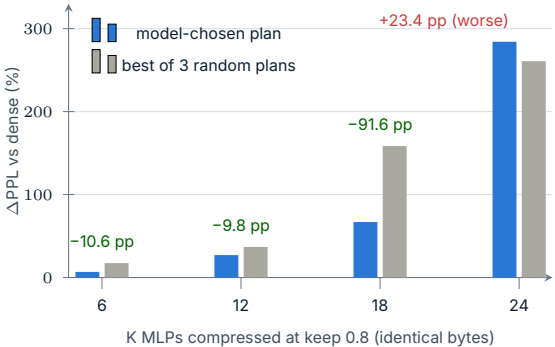
Component	Module	Status
Plan space, exact byte budget	loop/space.py	evaluated
Factorization machine (L-BFGS-B)	loop/fm.py	evaluated
CP-SAT / exact / greedy / SA / SB	loop/acquire.py	evaluated
Perplexity oracle, SVD + MPO	loop/oracle.py	evaluated
Paired bootstrap, Spearman	loop/stats.py	evaluated
SnapTwin prefill/decode model	twin/snaptwin.py	unit-tested
Placement QUBO (CPU/GPU/NPU)	qubo/planner.py	unit-tested
adb + llama-bench runners	bench/, scripts/	not yet run

- Qwen3-0.6B, 28 layers; MLP 1024×3072 (gate, up, down)
- keep 0.8 → rank 608 (79.2% of MLP params); keep 0.9 → rank 704 (91.7%)
- WikiText-2 test, context 512; second half of each chunk scored
- All results: PyTorch 2.14.1 + CUDA 12.6, fp32, NVIDIA GeForce RTX 3060 Laptop GPU
- Tests: 56 collected; 52 pass, 3 skip (optional torch/onnxruntime), 1 fails (smoke import of two subpackages missing from this snapshot)

Paths relative to isingedge/. Test run: python -m pytest tests/ (this snapshot).

RESULT 1 · MATCHED BYTES, FULL 584-CHUNK SPLIT

At equal bytes, planned layers beat random ones ($K \leq 18$)



−91.6 pp

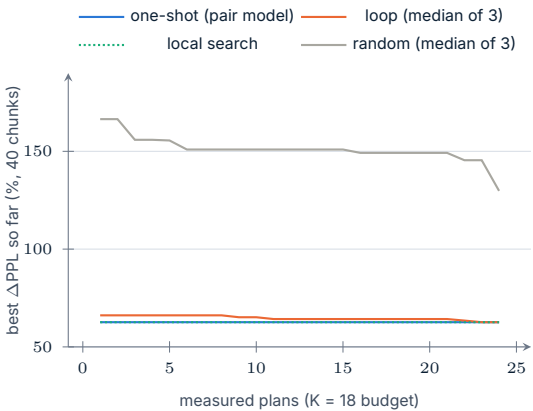
K = 18: +66.90% vs +158.51%; 95% CI [−94.03, −89.04] pp

- K = 6 / 12: −10.61 pp [−11.24, −9.95] / −9.78 pp [−10.59, −8.94]
- K = 24: model plan loses by +23.42 pp [+17.24, +29.55]: pair terms miss large-plan compounding
- Pair, additive and log-additive models pick the same layers here

Paired bootstrap (4,000 resamples), recomputed from results/sensitivity/qwen3-0.6b_plan_chunk_nll.csv; results/matched_bytes/summary.csv

RESULT 2 · CLOSED LOOP, 24 MEASURED PLANS PER RUN

The loop recovers the one-shot optimum; random trails by 20–85 pp



K	one-shot	random (3 seeds)
12	+25.45%	+45.68–47.50%
18	+64.52%	+115.37–149.85%

- Loop = one-shot plan in 3/3 seeds (K = 12) and 2/3 (K = 18); full-split ΔPPL above
- K = 18 seed 2: one swapped layer, −1.53 pp on the full split [−2.16, −0.90] but +0.83 pp on the 40-chunk objective
- Every acquisition solved to optimality by CP-SAT (48 binaries)

results/loop/evals.csv, results/loop/summary.csv. Plan budget in bytes of K MLPs at keep 0.8; options dense / keep 0.9 / keep 0.8 on layers 2–25.

EVIDENCE-BASED DESIGN CHOICES

What the measurements decided

15 / 72

rounds where greedy + swap descent matched the exact surrogate optimum (median gap 13.2% at K = 12, 4.1% at K = 18); ballistic-SB raw samples broke the constraints in 72/72 ⇒ exact CP-SAT

ρ = 0.52–0.81

surrogate rank correlation on 72 held-out random plans; it under-predicts damage (log-ratio bias −0.25 to −0.18) ⇒ re-measure every chosen plan

ρ = 0.64

Spearman of KL(dense‖compressed) vs single-layer ΔPPL (95% CI 0.35–0.83), below the 0.90 bar ⇒ perplexity stays the oracle

39.05 vs 53.17 PPL

3-core MPO vs SVD, layers 2–25 at equal params (91.7%), dense 21.88; accuracy only, no speed claim

results/loop/solver_gaps.csv, results/loop/calibration.csv, results/sensitivity/qwen3-0.6b_kl_proxy_summary.csv, results/compression/ppl.csv

HONEST STATUS

Limitations and next steps

Limitations

- No Snapdragon measurements in this repository: latency, memory and energy not measured; SnapTwin and the placement QUBO are unit-tested only
- One model (Qwen3-0.6B), one metric (WikiText-2 perplexity), one runtime (PyTorch fp32, laptop GPU); no task accuracy
- Post-training compression without healing: damage is large (+66.90% at K = 18)
- Pair model fails for large plans (K = 24); loop gains are small at 24 evaluations
- Plan-export and device scripts reference data not in this snapshot; one failing smoke test

Next steps

1. Run the scripted llama-bench sweeps (llama.cpp b11256) on a Snapdragon device over adb: CPU, GPU, NPU
2. Fit SnapTwin on device data; add latency terms to the same QUBO
3. Export plans as llama.cpp tensor overrides; measure TTFT, decode rate, memory
4. Add plan-size / higher-order terms; evaluate task accuracy and healing

TAKEAWAY

Compression plans must be measured and solved jointly

Which layers to compress is a combinatorial choice with compounding costs. Measuring whole plans and solving exactly under a byte budget finds much better plans than random choice, and shows where simple models fail.

−91.6 pp
vs best random plan at equal bytes (K = 18)

4.8×
measured vs summed single-layer damage (24 MLPs)

9.9×10^9
plans per budget, solved to optimality by CP-SAT

Measured: accuracy-side planner QUBO) **Not yet validated:** Snapdragon placement (SnapTwin + placement **Next:** on-device sweeps

Jenish Patel · Ising-Edge (MIT) · Qwen3 (Apache-2.0) · llama.cpp (MIT)